Python Assignment: Files, Exception Handling, Logging and Memory Management

Part A: Theory Questions | Part B: Practical Questions

Part A: Theory Questions

1. What is the difference between interpreted and compiled languages?

Compiled language	Interpreted language
Whole source code is translated into machine code by a compiler before execution.	Source code is executed line by line by an interpreter at run time.
Produces a separate executable file.	No separate executable is produced.
Faster execution; errors are reported at compile time, before the program runs.	Slower execution; errors are found only when the faulty line is reached.
Examples: C, C++, Rust, Go.	Examples: Python, JavaScript, Ruby, PHP.

Note: Python is usually called interpreted, but the source is first compiled to bytecode (.pyc), which is then run by the Python Virtual Machine (PVM).

2. What is exception handling in Python?

An **exception** is an error that occurs while a program is running and disrupts its normal flow (for example dividing by zero or opening a missing file). **Exception handling** is the mechanism of catching these errors with `try`, `except`, `else` and `finally` so that the program can deal with them gracefully instead of crashing.

Code:

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
print("Program continues normally")
```

Output:

```
Cannot divide by zero
Program continues normally
```

3. What is the purpose of the finally block in exception handling?

The `finally` block contains code that is **always executed**, whether or not an exception occurred and whether or not it was caught. It is mainly used for cleanup work such as closing files, closing database connections or releasing resources.

Code:

```
try:
    num = int("abc")
except ValueError:
    print("Invalid number")
finally:
    print("Finally block always runs")
```

Output:

```
Invalid number
Finally block always runs
```

4. What is logging in Python?

Logging is the process of recording messages about what a program is doing (events, warnings, errors) while it runs. Python provides the built-in `logging` module for this. Log messages can be shown on the

console or saved to a file, together with a timestamp, severity level and message, which helps in debugging and monitoring.

Code:

```
import logging, sys
logging.basicConfig(stream=sys.stdout, level=logging.INFO,
                    format="%(levelname)s - %(message)s", force=True)
logging.info("Program started")
logging.warning("Disk space is low")
```

Output:

```
INFO - Program started
WARNING - Disk space is low
```

5. What is the significance of the __del__ method in Python?

`__del__` is a special method (destructor) that is called automatically just before an object is destroyed, i.e. when its reference count becomes zero and it is garbage collected. It is used to release resources such as files, network connections or database handles that the object holds. It should be used carefully because the exact time of its call is not guaranteed.

Code:

```
class Demo:
    def __init__(self, name):
        self.name = name
        print(self.name, "created")
    def __del__(self):
        print(self.name, "destroyed")

d = Demo("Object A")
del d
print("End of program")
```

Output:

```
Object A created
Object A destroyed
End of program
```

6. What is the difference between import and from ... import in Python?

import module	from module import name
Imports the whole module.	Imports only the specified names (functions, classes, variables).
Names are accessed with the module prefix: math.sqrt(16).	Names are used directly: sqrt(16).
Avoids name clashes.	Can overwrite existing names in your program.

Code:

```
import math
print(math.sqrt(16))

from math import sqrt, pi
print(sqrt(25), round(pi, 2))
```

Output:

```
4.0
5.0 3.14
```

7. How can you handle multiple exceptions in Python?

There are three ways: (a) write **multiple except blocks**, one for each exception type; (b) catch several exceptions in **one except block using a tuple**; (c) use a generic `except Exception` as the last block to catch anything else.

Code:

```
def divide(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        return "Division by zero"
    except (TypeError, ValueError):
        return "Invalid input type"
    except Exception as e:
        return "Other error: " + str(e)

print(divide(10, 2))
print(divide(10, 0))
print(divide(10, "a"))
```

Output:

```
5.0
Division by zero
Invalid input type
```

8. What is the purpose of the with statement when handling files in Python?

The `with` statement is a **context manager**. It opens the file and **automatically closes it** when the block ends, even if an exception occurs inside the block. This avoids resource leaks and removes the need to call `close()` manually.

Code:

```
with open("demo.txt", "w") as f:
    f.write("Hello")
print("File closed automatically:", f.closed)
```

Output:

```
File closed automatically: True
```

9. What is the difference between multithreading and multiprocessing?

Multithreading	Multiprocessing
Multiple threads run inside one process and share the same memory.	Multiple separate processes run, each with its own memory space.
Limited by the GIL (Global Interpreter Lock) in CPython, so no true parallelism for CPU-bound code.	Each process has its own GIL, so real parallelism on multiple CPU cores.
Lightweight, quick to create, easy to share data.	Heavier, slower to start, data sharing needs queues or pipes.
Best for I/O-bound tasks (network calls, file reading).	Best for CPU-bound tasks (heavy calculations).
Module: threading	Module: multiprocessing

Code:

```
import threading
from multiprocessing import Process

def task(n):
    print("Task", n)

t = threading.Thread(target=task, args=(1,))   # multithreading
t.start(); t.join()

if __name__ == "__main__":
    pr = Process(target=task, args=(2,))        # multiprocessing
    pr.start(); pr.join()
```

10. What are the advantages of using logging in a program?

- Helps in **debugging** and finding the cause of errors, especially after the program has run.
- Provides a **permanent record** of events in a file, unlike print statements which disappear.

- Supports **severity levels** (DEBUG to CRITICAL), so messages can be filtered.
- Adds useful details automatically: timestamp, module name, line number.
- Output can be sent to **multiple destinations** (console, file, email, network).
- Logging can be switched on/off or changed by configuration without editing the code, unlike print().

11. What is memory management in Python?

Memory management is the process of **allocating memory** to objects when they are created and **freeing it** when they are no longer needed. In Python this is automatic. A private heap holds all objects and data structures, and the Python memory manager handles allocation. Python uses **reference counting** together with a **cyclic garbage collector** to reclaim unused memory, so the programmer does not need to free memory manually as in C.

12. What are the basic steps involved in exception handling in Python?

- **Step 1:** Identify the code that may raise an exception.
- **Step 2:** Place that code inside a `try` block.
- **Step 3:** Write one or more `except` blocks to catch and handle specific exceptions.
- **Step 4 (optional):** Add an `else` block for code that should run only when no exception occurs.
- **Step 5 (optional):** Add a `finally` block for cleanup code that must always run.
- **Step 6 (optional):** Use `raise` to raise an exception manually where needed.

Code:

```
try:
    x = int("25")
except ValueError:
    print("Not a number")
else:
    print("Converted:", x)
finally:
    print("Done")
```

Output:

```
Converted: 25
Done
```

13. Why is memory management important in Python?

- **Prevents memory leaks:** unused objects are released so the program does not keep consuming RAM.
- **Better performance:** efficient use of memory makes programs faster and avoids swapping to disk.
- **Prevents crashes:** running out of memory causes MemoryError or the program being killed.
- **Scalability:** large applications (data processing, web servers) must handle big data with limited memory.
- **Resource efficiency:** especially important on servers, cloud systems and small devices.

14. What is the role of try and except in exception handling?

The `try` block holds the code that might cause an error. If an exception occurs inside it, Python immediately stops executing the `try` block and jumps to the matching `except` block, which contains the code to handle the error. If no exception occurs, the `except` block is skipped. Together they stop the program from terminating abruptly.

Code:

```
try:
    age = int("twenty")
except ValueError as e:
    print("Error caught:", e)
```

Output:

```
Error caught: invalid literal for int() with base 10: 'twenty'
```

15. How does Python's garbage collection system work?

Python uses two mechanisms:

- **Reference counting:** each object keeps a count of how many references point to it. When the count drops to 0, the object is deleted immediately and its memory is freed.
- **Cyclic garbage collector (gc module):** reference counting cannot free objects that refer to each other in a cycle (A refers to B and B refers to A). The gc module periodically detects such unreachable cycles and frees them. Objects are placed in **three generations** (0, 1, 2); new objects are in generation 0 and are checked most often, while objects that survive move to older generations that are checked less often.

Code:

```
import sys, gc
a = [1, 2, 3]
b = a
print("Reference count:", sys.getrefcount(a))   # a, b and the argument

class Node:
    pass
n1, n2 = Node(), Node()
n1.other, n2.other = n2, n1       # circular reference
del n1, n2
print("Unreachable objects collected:", gc.collect() > 0)
```

Output:

```
Reference count: 3
Unreachable objects collected: True
```

16. What is the purpose of the else block in exception handling?

The `else` block runs **only if the `try` block completes without raising any exception**. It separates the code that should run on success from the code that is being guarded, which keeps the `try` block short and avoids accidentally catching exceptions from the success code.

Code:

```
try:
    num = int("10")
except ValueError:
    print("Invalid input")
else:
    print("No error, number is", num)
```

Output:

```
No error, number is 10
```

17. What are the common logging levels in Python?

Level	Value	Used for
DEBUG	10	Detailed information for diagnosing problems.
INFO	20	Confirmation that things work as expected.
WARNING	30	Something unexpected happened or may happen soon (default level).
ERROR	40	A serious problem; some function could not be performed.
CRITICAL	50	Very serious error; the program may not be able to continue.

Only messages at or above the configured level are recorded.

18. What is the difference between os.fork() and multiprocessing in Python?

os.fork()	multiprocessing
Low-level call that creates a child copy of the current process.	High-level module with Process, Pool, Queue, Pipe, etc.
Available only on Unix/Linux/macOS, not on Windows.	Cross-platform (Windows, Linux, macOS).
Returns 0 in the child and the child's PID in the parent; the programmer manages everything.	Provides easy APIs for starting, joining and communicating between processes.
No built-in data sharing or synchronization.	Built-in queues, locks, shared memory and pools.

Code:

```
import os
from multiprocessing import Process

pid = os.fork()                # Unix only
if pid == 0:
    print("Child process")
else:
    print("Parent process, child PID =", pid)

def work():
    print("Running in a separate process")
if __name__ == "__main__":
    p = Process(target=work); p.start(); p.join()
```

19. What is the importance of closing a file in Python?

- Makes sure data buffered in memory is **written (flushed) to the disk**, otherwise data may be lost.
- **Frees system resources** such as file handles; operating systems limit how many files can be open.
- Avoids **file locking** problems so other programs can access the file.
- Prevents memory leaks and data corruption. Using `with open(...)` closes the file automatically.

Code:

```
f = open("close_demo.txt", "w")
f.write("data")
print("Closed?", f.closed)
f.close()
print("Closed?", f.closed)
```

Output:

```
Closed? False
Closed? True
```

20. What is the difference between file.read() and file.readline() in Python?

file.read()	file.readline()
Reads the whole file (or n characters if given) as a single string.	Reads only one line at a time, including the newline character.
Uses more memory for large files.	Memory efficient for large files.
Returns an empty string at end of file.	Returns an empty string at end of file.

Code:

```
with open("lines.txt", "w") as f:
    f.write("Line one\nLine two\nLine three\n")

with open("lines.txt") as f:
    print(repr(f.readline()))   # first line only
    print(repr(f.read()))       # the rest of the file
```

Output:

```
'Line one\n'
'Line two\nLine three\n'
```

21. What is the logging module in Python used for?

The `logging` module is the standard library module used to **record events that happen while a program runs**. It lets programmers create log messages of different severity levels (DEBUG, INFO, WARNING, ERROR, CRITICAL), format them (time, level, message), and send them to different destinations using handlers such as `StreamHandler`, `FileHandler` and `RotatingFileHandler` (rotating log files). It is used for debugging, monitoring, auditing and troubleshooting applications.

22. What is the os module in Python used for in file handling?

The `os` module provides functions to interact with the operating system, so files and folders can be managed from Python. Common functions:

Function	Purpose
os.path.exists(path), os.path.isfile(path)	Check whether a file or folder exists.
os.path.getsize(path)	Get the file size in bytes.
os.remove(path), os.rename(old, new)	Delete or rename a file.
os.mkdir(name), os.listdir(path)	Create a directory, list directory contents.
os.getcwd(), os.path.join(a, b)	Get the current directory, build file paths.

Code:

```
import os
open("os_demo.txt", "w").write("hello")
print(os.path.exists("os_demo.txt"))
print(os.path.getsize("os_demo.txt"), "bytes")
os.rename("os_demo.txt", "os_renamed.txt")
os.remove("os_renamed.txt")
print(os.path.exists("os_renamed.txt"))
```

Output:

```
True
5 bytes
False
```

23. What are the challenges associated with memory management in Python?

- **Memory leaks:** objects that are still referenced (global variables, caches, lists) are never freed.
- **Circular references:** need the cyclic garbage collector, and cycles with __del__ are harder to clean.
- **High memory use:** Python objects have extra overhead (even a small integer takes several bytes more than in C).
- **No manual control:** the programmer cannot decide exactly when memory is freed; garbage collection timing is unpredictable.
- **Memory fragmentation** and large objects (big lists, DataFrames) can exhaust RAM.
- **Garbage collection overhead:** periodic collection can cause small performance pauses.
- **Hard to debug:** finding which object holds the memory needs tools such as tracemalloc or memory_profiler.

24. How do you raise an exception manually in Python?

Use the `raise` keyword followed by an exception class (optionally with a message). This is useful for enforcing validation rules. Custom exceptions can be created by inheriting from `Exception`.

Code:

```
def check_age(age):
    if age < 18:
        raise ValueError("Age must be at least 18")
    return "Access granted"

try:
```

```
    print(check_age(20))
    print(check_age(15))
except ValueError as e:
    print("Error:", e)
```

Output:

```
Access granted
Error: Age must be at least 18
```

25. Why is it important to use multithreading in certain applications?

- **Better responsiveness:** a GUI or web app stays responsive while a long task runs in a background thread.
- **Faster I/O-bound tasks:** while one thread waits for network, disk or database, others continue working (e.g. downloading many files at once, web scraping, handling many client requests).
- **Shared memory:** threads share the same data, so communication is easy and cheap.
- **Lightweight:** threads use fewer resources than processes.
- **Better resource use:** keeps the CPU busy instead of idle waiting.
- **Note:** because of the GIL, multithreading does not speed up CPU-heavy work in CPython; multiprocessing is better there.

Part B: Practical Questions

Note: Practical Question 1 creates the file notes.txt, which later programs read. Run Question 1 first.

1. How can you open a file for writing in Python and write a string to it?

Code:

```
with open("notes.txt", "w") as f:
    f.write("Python is easy to learn.\nPython is powerful.\nFile handling in Python is simple.\n")
print("File written successfully")
```

Output:

```
File written successfully
```

2. Write a Python program to read the contents of a file and print each line

Code:

```
with open("notes.txt", "r") as f:
    for line in f:
        print(line.strip())
```

Output:

```
Python is easy to learn.
Python is powerful.
File handling in Python is simple.
```

3. How would you handle a case where the file doesn't exist while opening it for reading?

Code:

```
try:
    with open("missing.txt", "r") as f:
        print(f.read())
except FileNotFoundError:
    print("Error: The file does not exist.")
```

Output:

```
Error: The file does not exist.
```

4. Write a script that reads from one file and writes its content to another file

Code:

```
with open("notes.txt", "r") as source, open("copy.txt", "w") as target:
    target.write(source.read())

with open("copy.txt") as f:
    print("Content of copy.txt:")
    print(f.read())
```

Output:

```
Content of copy.txt:
Python is easy to learn.
Python is powerful.
File handling in Python is simple.
```

5. How would you catch and handle a division by zero error?

Code:

```
try:
    a = 10
    b = 0
    print(a / b)
except ZeroDivisionError:
    print("Error: Division by zero is not allowed")
```

Output:

```
Error: Division by zero is not allowed
```

6. Program that logs an error message to a log file when a division by zero occurs

Code:

```
import logging

logging.basicConfig(filename="division_error.log", level=logging.ERROR,
                    format="%(asctime)s - %(levelname)s - %(message)s", force=True)
try:
    result = 10 / 0
except ZeroDivisionError:
    logging.error("Division by zero attempted")
    print("Error logged to division_error.log")

print("--- Content of division_error.log ---")
print(open("division_error.log").read())
```

Output:

```
Error logged to division_error.log
--- Content of division_error.log ---
2026-10-26 16:41:15,938 - ERROR - Division by zero attempted
```

7. How do you log information at different levels (INFO, ERROR, WARNING)?

Code:

```
import logging, sys

logging.basicConfig(stream=sys.stdout, level=logging.DEBUG,
                    format="%(levelname)s: %(message)s", force=True)
logging.debug("Debug message")
logging.info("Information message")
logging.warning("Warning message")
logging.error("Error message")
logging.critical("Critical message")
```

Output:

```
DEBUG: Debug message
INFO: Information message
WARNING: Warning message
ERROR: Error message
CRITICAL: Critical message
```

8. Write a program to handle a file opening error using exception handling

Code:

```
filename = "data_file.txt"
try:
    f = open(filename, "r")
except FileNotFoundError:
    print("File not found:", filename)
except PermissionError:
    print("No permission to open:", filename)
except OSError as e:
    print("Could not open file:", e)
else:
    print(f.read())
    f.close()
finally:
    print("File operation attempted")
```

Output:

```
File not found: data_file.txt
File operation attempted
```

9. How can you read a file line by line and store its content in a list?

Code:

```
lines = []
```

```
with open("notes.txt", "r") as f:
    for line in f:
        lines.append(line.strip())
print(lines)

# shorter alternative
with open("notes.txt") as f:
    lines2 = f.read().splitlines()
print(lines2)
```

Output:

```
['Python is easy to learn.', 'Python is powerful.', 'File handling in Python is simple.']
['Python is easy to learn.', 'Python is powerful.', 'File handling in Python is simple.']
```

10. How can you append data to an existing file?

Code:

```
with open("notes.txt", "a") as f:     # "a" = append mode
    f.write("This line was appended.\n")

with open("notes.txt") as f:
    print(f.read())
```

Output:

```
Python is easy to learn.
Python is powerful.
File handling in Python is simple.
This line was appended.
```

11. try-except to handle an error when accessing a dictionary key that doesn't exist

Code:

```
student = {"name": "Riya", "age": 20}
try:
    print(student["marks"])
except KeyError:
    print("Error: The key 'marks' does not exist in the dictionary")

# alternative using get()
print(student.get("marks", "Not available"))
```

Output:

```
Error: The key 'marks' does not exist in the dictionary
Not available
```

12. Program with multiple except blocks to handle different types of exceptions

Code:

```
def process(value):
    try:
        number = int(value)
        result = 100 / number
        print("Result:", result)
    except ValueError:
        print("ValueError: '%s' is not a valid integer" % value)
    except ZeroDivisionError:
        print("ZeroDivisionError: cannot divide by zero")
    except TypeError:
        print("TypeError: unsupported type")
    finally:
        print("-- processed --")

process("5")
process("abc")
process("0")
process(None)
```

Output:

```
Result: 20.0
-- processed --
ValueError: 'abc' is not a valid integer
-- processed --
ZeroDivisionError: cannot divide by zero
-- processed --
TypeError: unsupported type
-- processed --
```

13. How would you check if a file exists before attempting to read it?

Code:

```
import os

filename = "notes.txt"
if os.path.exists(filename):
    with open(filename) as f:
        print("File found. First line:", f.readline().strip())
else:
    print("File does not exist")

print(os.path.isfile("unknown.txt"))
```

Output:

```
File found. First line: Python is easy to learn.
False
```

14. Program using the logging module to log both informational and error messages

Code:

```
import logging

logging.basicConfig(filename="app.log", level=logging.INFO,
                    format="%(asctime)s - %(levelname)s - %(message)s", force=True)

logging.info("Program started")
try:
    value = int("abc")
except ValueError as e:
    logging.error("Conversion failed: %s", e)
logging.info("Program finished")

print(open("app.log").read())
```

Output:

```
2026-10-26 16:41:15,941 - INFO - Program started
2026-10-26 16:41:15,941 - ERROR - Conversion failed: invalid literal for int() with base 10:
'abc'
2026-10-26 16:41:15,941 - INFO - Program finished
```

15. Program that prints the content of a file and handles the case when the file is empty

Code:

```
open("empty.txt", "w").close()          # create an empty file

def show_file(name):
    with open(name) as f:
        content = f.read()
    if content == "":
        print(name, "-> The file is empty")
    else:
        print(name, "->")
        print(content)

show_file("empty.txt")
show_file("notes.txt")
```

Output:

```
empty.txt -> The file is empty
notes.txt ->
Python is easy to learn.
Python is powerful.
File handling in Python is simple.
This line was appended.
```

16. Demonstrate memory profiling to check the memory usage of a small program

Using the built-in `tracemalloc` module (the `memory_profiler` package with the `@profile` decorator can also be used, but must be installed with pip).

Code:

```
import tracemalloc

tracemalloc.start()
data = [i * 2 for i in range(100000)]       # the small program
current, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()

print("Current memory: %.2f MB" % (current / 1024 / 1024))
print("Peak memory   : %.2f MB" % (peak / 1024 / 1024))
```

Output:

```
Current memory: 3.81 MB
Peak memory   : 3.81 MB
```

17. Create and write a list of numbers to a file, one number per line

Code:

```
numbers = [10, 20, 30, 40, 50]
with open("numbers.txt", "w") as f:
    for n in numbers:
        f.write(str(n) + "\n")

with open("numbers.txt") as f:
    print(f.read())
```

Output:

```
10
20
30
40
50
```

18. Basic logging setup that logs to a file with rotation after 1 MB

Code:

```
import logging, os
from logging.handlers import RotatingFileHandler

logger = logging.getLogger("rotating_demo")
logger.setLevel(logging.INFO)
handler = RotatingFileHandler("rotate.log", maxBytes=1024 * 1024, backupCount=3)  # 1 MB
handler.setFormatter(logging.Formatter("%(asctime)s - %(levelname)s - %(message)s"))
logger.addHandler(handler)

for i in range(20000):                        # enough messages to exceed 1 MB
    logger.info("Log message number %d - testing log rotation", i)

print(sorted(f for f in os.listdir(".") if f.startswith("rotate.log")))
```

Output:

```
['rotate.log', 'rotate.log.1']
```

When `rotate.log` grows beyond 1 MB, it is renamed to `rotate.log.1` and a new `rotate.log` is started. Up to `backupCount` backup files are kept; the oldest is deleted.

19. Program that handles both IndexError and KeyError using a try-except block

Code:

```
my_list = [1, 2, 3]
my_dict = {"a": 1}

for action in ("index", "key"):
    try:
        if action == "index":
            print(my_list[10])
        else:
            print(my_dict["z"])
    except (IndexError, KeyError) as e:
        print(type(e).__name__, "caught:", e)
```

Output:

```
IndexError caught: list index out of range
KeyError caught: 'z'
```

20. How would you open a file and read its contents using a context manager?

Code:

```
with open("notes.txt", "r") as file:
    content = file.read()
    print(content)
print("File closed:", file.closed)
```

Output:

```
Python is easy to learn.
Python is powerful.
File handling in Python is simple.
This line was appended.

File closed: True
```

21. Program that reads a file and prints the number of occurrences of a specific word

Code:

```
import re

word = "python"
with open("notes.txt", "r") as f:
    text = f.read()

count = len(re.findall(r"\b" + re.escape(word) + r"\b", text, re.IGNORECASE))
print("The word '%s' occurs %d times" % (word, count))
```

Output:

```
The word 'python' occurs 3 times
```

22. How can you check if a file is empty before attempting to read its contents?

Code:

```
import os

open("empty.txt", "w").close()
for name in ("empty.txt", "notes.txt"):
    if os.path.getsize(name) == 0:
        print(name, "is empty")
    else:
        print(name, "has", os.path.getsize(name), "bytes, safe to read")
```

Output:

```
empty.txt is empty
notes.txt has 104 bytes, safe to read
```

23. Program that writes to a log file when an error occurs during file handling

Code:

```
import logging

logging.basicConfig(filename="file_errors.log", level=logging.ERROR,
                    format="%(asctime)s - %(levelname)s - %(message)s", force=True)
try:
    with open("nonexistent_file.txt", "r") as f:
        data = f.read()
except FileNotFoundError as e:
    logging.error("File handling error: %s", e)
    print("An error occurred. Check file_errors.log")

print("--- Content of file_errors.log ---")
print(open("file_errors.log").read())
```

Output:

```
An error occurred. Check file_errors.log
--- Content of file_errors.log ---
2026-10-26 16:41:16,808 - ERROR - File handling error: [Errno 2] No such file or directory:
'nonexistent_file.txt'
```